TIM MENZIES

SIMPLE AIN’T STUPID

Preface

Much of what we do in software engineering and AI comes from a small number of core ideas, wrapped in vast amounts of ceremony. This book tries to show the core without the ceremony. So we set ourselves a task: teach 200 heuristics, tips, tricks, and traps from scripting, SE, and AI, using as little code as possible. Chasing that task produced EZR (an incremental active learner whose conclusions are small decision trees), about 400 lines of Python with no dependencies beyond the standard library (Menzies and Srinivasan 2026). We do not say EZR is the only good toolkit. It is the one this hunt left us, and it is small enough to read. Around it we hang 123 lessons. Many appear more than once, seen from different heights, so the count of sightings runs past 200.

In the age of large language models, it is said that software is free: ask, and code appears. We agree, in part. Most software is now cheap. Good software is very, very expensive. Good software is maintainable, clear enough for a stranger to understand, and built so the next feature has somewhere to go. Good software is not rushed out the door and left to gather dirt. It is inspected with care, cleaned regularly, and refactored, over and over, to be made simpler. That work is the expensive part, and that work is what this book teaches.

Every chapter follows the same ritual: (a) set a seed; (b) run a small program; (c) paste its output, untouched; (d) end in an assert. Nothing you read here was typed from memory. The build tool re-runs every example and fails if the output drifts.[1]

[1] If you find a transcript in this book that does not reproduce, that is a bug. Please report it.

How to read this book

Read Chapter 1, then skip Chapter 2. That chapter places this book against fifty years of prior work and it will mean more after you have met the code. Come back to it late, or never.

Chapters 3 to 5 build the substrate: a little Python, a little maths, then columns, tables, and distance. All later chapters stand on these three. After that, each chapter is one application with a fun name

and a serious bracket (e.g. "The Bouncer (anomaly detection)"). The bracketed term is the index entry that joins that chapter to the standard literature. Read the applications in any order.

You need Python 3.12 or later, curl, and make. You do not need pip. The number of packages this book installs is $0 + 0 = 0$. Example data comes from MOOT (a public collection of 120+ multi-objective SE optimization tasks)[2] and one table runs through most demos: auto93, 398 cars, where we want to minimize weight while maximizing acceleration and miles per gallon.

[2] `github.com/timm/moot`; fetched by `make data`.

The Substrate

Simple Ain't Stupid

Much recent press says that developers no longer need to read code. The creator of Node.js suggests the era of human-written code is ending. Nvidia's CEO advises against learning to code at all (Menzies and Srinivasan 2026). The claim behind the phrasing is always the same: AI is the new compiler, and nobody reads what a compiler emits.

We disagree. This book is our evidence.

The evidence is 400 lines of Python that implement Naive Bayes, k-means clustering, decision trees, anomaly detection, active learning, and multi-objective optimization on one shared substrate. Tested on 120+ tasks from the MOOT repository, these tiny tools perform as well as or better than SHAP, LIME, and the SMAC3 optimizer, while running 500 times faster than SMAC3 and using orders of magnitude fewer labels (Menzies and Srinivasan 2026). That result was found by reading: years of taking learners apart, noticing that distant parts of different algorithms were the same part, and deleting the copies. Reading code is not nostalgia. It is a research method, and it works.

The axiom

One economic fact organizes everything that follows. Tables of data are cheap; *labels* are dear. Anyone can list 10,000 configuration options. Knowing which one compiles fastest costs a compile per option. Hence the question this book asks, over and over: how few labels buy a good row, plus an explanation of why it is good?

Our running example is auto93: 398 cars, 8 columns. Four columns are cheap to observe (cylinders, engine volume, model year, origin). Three are goals we must pay to know: minimize Lbs-, maximize Acc+ and Mpg+. One (HpX) we ignore. That is $4 + 3 + 1 = 8$. When we say a method is good, we mean it finds near-best cars after paying for few labels.

The ritual

Every claim in this book can be re-run from a seed, and any claim that cannot is not in this book. Each demo (a) resets the random stream; (b) runs; (c) prints; (d) asserts. The printed output is captured by the build tool at build time, so what you read is what the code did, on the day this book was compiled. Chapter 15 shows the statistics that police the stronger claims: no "X beats Y" without effect size and significance agreeing.

The map

Part I (Chapters 3 to 5) builds the substrate: cells, columns, tables, distance. Part II reads the world: six applications that predict, detect, monitor, diagnose, triage, and explain. Part III changes the world: eight applications that optimize, plan, repair, generate, compress, negotiate, and learn on streams. Part IV earns trust: certification by statistics, and the onboarding of an AI colleague. Chapter 2 relates all this to prior work. Skip it for now.

One convention, used throughout. Where a lesson has a canonical name, that name appears in bold at first use (e.g. **SSOT**, single source of truth). Where a lesson has no canonical name, none is invented. The unnamed ones are the new ones.

Before Us (Skip This on First Read)

This chapter has one job: to say what is old here, so that what is new stands out honestly. It cites much and proves nothing. Come back after Part II.

Four crowded neighborhoods

Code with commentary, on one system, has a patron saint: Lions' commentary on UNIX, the complete Edition 6 source plus a reading of it, bootlegged for decades and still used in teaching (Lions 1996). Spinellis later made code reading a discipline of its own (Spinellis 2003). We take from Lions the core structural move: print the system once, then tour it repeatedly at different altitudes.

Language books with practicals form the second neighborhood. Norvig's *PAIP* taught classic AI through small idiomatic Common Lisp programs (Norvig 1992). Seibel's *Practical Common Lisp* taught a language by building a spam filter, an MP3 database, and other things a reader might actually want (Seibel 2005). Our Part II and Part III copy Seibel's test directly: every chapter is named for a want, and the machinery hides behind it.

Small-programs anthologies are the third. *500 Lines or Less* walks 26 programs by 26 authors, each solving a canonical problem in under 500 lines (Brown and DiBernardo 2016). Wilson's *Software Design by Example* does the same solo (Wilson 2024). These books share our size discipline. They do not share our substrate: their programs have nothing in common, so their lessons cannot compose. Ours do, since every application here reuses the same 400 lines.

Heuristics catalogs are the fourth: Hunt and Thomas (Hunt and Thomas 1999), Glass's facts and fallacies (Glass 2002), Ousterhout (Ousterhout 2018). These argue by anecdote and citation. We argue by flag: when this book says "always keep a baseline", that baseline is one command-line option away, and you can run the ablation yourself.

Each neighborhood is crowded. Their intersection, as far as we can tell, is empty. That intersection is this book.

The grid and the ladder

Buse and Zimmermann once organized software analytics as a 3-by-3 grid: exploration, analysis, and experimentation crossed with past, present, and future (Buse and Zimmermann 2012). It was, and is, a useful catalog of what managers ask for. Yet we read the grid differently. Its three rows are the three rungs of Pearl's ladder of causation: seeing, explaining, and imagining alternatives (Pearl and Mackenzie 2018). Its three columns are a clock. And a clock is a presentation dimension, not a reasoning one. Chapters 8, 9, and 10 of this book make that point in code: planning, diagnosis, and repair turn out to be one tree-difference mechanism pointed at the future, the past, and the imperative. Time multiplies interfaces. It does not multiply mechanisms. Hence, in this book, streaming is a crosscutting section in many chapters rather than a wing of the taxonomy.

Tasks, not tenses

For the taxonomy of tasks we reach further back, to knowledge engineering. Clancey observed that expert systems mostly perform *heuristic classification* (Clancey 1985), and the CommonKADS school cataloged the task types (Schreiber et al. 2000): analysis tasks (predict, monitor, diagnose, classify, explain) and synthesis tasks (plan, design, configure, repair). That catalog is finite. Part II covers the analysis tasks. Part III covers the synthesis tasks, plus the modern additions (compress, negotiate, stream). We know of no other short book that runs the whole KADS catalog on one substrate.

There is one rung the old catalogs lack. Neither the grid nor the ladder nor KADS has a place for *certification*: establishing that the seeing and the doing can be trusted by another person, another runtime, or another kind of colleague. Buse and Zimmermann tuck significance testing into a corner cell as a bullet point. We give it Part IV. The 2026 problem, we will argue, lives on that rung, and it costs about sixty lines.

What is actually new

Firstly, the substrate: many seemingly different learners shown as one grouping idea at different granularities, in runnable form. Secondly, the

receipts: heuristics with flags and seeds, not heuristics with anecdotes. Thirdly, the last two chapters: a statistics gate and an agent-onboarding process, both demonstrated from one repository's own history. To our knowledge the third item exists in no prior book at all. That said, it may date the fastest. We accept the trade.

A Little Python

This chapter shows the Python that the rest of the book assumes. Not all of Python. Just the dozen moves that let 400 lines carry twenty algorithms. Readers fluent in the language should skim the transcripts and move on.

Cells, and asking forgiveness

Csv cells arrive as strings. One function coerces each cell: "23" becomes an int, "-1e2" a float (csv cells can hide exponents), "True" and "False" become bools, "?" stays as our mark for missing, and anything else stays text.

```
def thing(s): # string to int, float, bool, or string
  for fn in (int, float):
    try: return fn(s)
    except ValueError: pass
  s = s.strip()
  return {"True": True, "False": False}.get(s, s)
```

Note the shape of the type test. We do not inspect the string to decide if it is a number. We try the conversion and catch the failure. Python calls this **EAFP** (easier to ask forgiveness than permission), as opposed to **LBYL** (look before you leap). EAFP is shorter here and, more importantly, it delegates the definition of "looks like an int" to the one authority that matters, which is `int` itself.

```
$ python3 src/lib_eg.py thing
[23, 3.14, -100.0, True, False, '?', 'ab']
```

One struct, one settings object

The whole book uses one generic record type: a class `o` whose instances are dot-accessible bags of slots, and whose repr prints those slots sorted.

```
class o:
  def __init__(i, **d): i.__dict__.update(**d)
  def __repr__(i):
    return "{" + " ".join(":%s %s" % (k, v)
      for k, v in sorted(i.__dict__.items())
      if k[0] != "_") + "}"
```

Two Python facts do the work: (a) every object carries a `__dict__` of its slots, so one line of reflection prints anything; (b) `__repr__` is a protocol, so every object in this book can appear in a transcript. Also, all settings live in a single instance called `the`, defined in about.py and nowhere else. That file is this book's first sighting of **SSOT** (single source of truth): one place to look, one place to change.

```
$ python3 src/lib_eg.py the
{:few 128 :file data/auto93.csv :p 2 :seed 1234567891 :stop 32}
```

The command line can override any knob, because the flag parser walks `vars(the)` and matches names. Watch the same test, run with `--p=1`:

```
$ python3 src/lib_eg.py the --p=1
{:few 128 :file data/auto93.csv :p 1 :seed 1234567891 :stop 32}
```

No parser was written for that flag. The settings object is the parser's schema, by reflection. Hence a new knob costs one line, in one file.

Streams, sorts, and transposes

Files are read by a generator, so no table is ever loaded twice or held whole when a stream will do:

```
def csv(file): # stream rows of coerced cells
  with open(file) as f:
    for line in f:
      if (line := line.split("%")[0].strip()):
        yield [thing(s) for s in line.split(",")]
```

The `yield` makes `csv` lazy: callers pull one row at a time. Later chapters lean hard on this laziness (e.g. the streaming chapter treats an unbounded source exactly like a file). Three more idioms appear on nearly every page of this book, so we test them once here: sorting by a computed key, never in place; transposing a table with `zip(*rows)`; and list comprehensions as the default loop.

```
$ python3 src/lib_eg.py idioms
[[3, 30], [2, 20], [1, 10]]
[(1, 3, 2), (10, 30, 20)]
```

Dispatch by name

Every code file in this book ships with a matching `_eg` file of demos. Each demo runs by its bare name from the command line. The trick is four lines of reflection. The caller hands over its `globals()` as `g`, a plain dict mapping names to the things they name. On any dict, `g.get(key, default)` returns `g[key]` if the key is present, else the default. Hence: look up `"test_" + word`, falling back to a function that just complains.

```
def main(g): # for each bare word w, run test_w, seeded
  cli(the.__dict__)
  todo = [s for s in sys.argv[1:]
          if not s.startswith("-")] or ["all"]
  for word in todo:
    random.seed(the.seed)
    g.get("test_" + word,
          lambda: print("?", word, "(no such test)"))()
```

Note that the seed is reset before every test. Hence any demo reproduces in isolation, in any order, on any machine. This one habit does more for reproducibility than any tool we know, and it costs one line.

Lessons sighted

EAFP and **LBYL**; **SSOT**; duck typing (previewed; defined properly in Chapter 5); generators and laziness; reflection via `vars` and `globals`; sort-by-key; `zip(*rows)`; the seeded-demo ritual. Just to repeat a point made above: none of this is advanced Python. That is the point. Relentless basics, then one or two sharp tricks.

A Little Maths

Six pieces of maths run the whole book: two summaries, a rescaling, a distance, a random stream, and a trust test. None needs more than high-school algebra. Each gets a demo here and a citation for readers who want the long form. But first, some words.

Four kinds of column, two survive

Statistics sorts columns into four scales, by what you may do to their values: nominal (names; test equality, nothing else), ordinal (ranks; also sort), interval (also subtract), and ratio (also divide) (Stevens 1946). The acronym is **NOIR**. This book keeps the two ends. Names become `Sym` and numbers become `Num`; the middle two get treated as one or the other.

```
def Sym(name="", at=0): # summary of a symbolic column
  return o(it=Sym, at=at, name=name, n=0, has={})


def Num(name="", at=0): # summary of a numeric column
  return o(it=Num, at=at, name=name, n=0, mu=0, m2=0,
           heaven=0 if name.endswith("-") else 1)
```

Every column summary answers the same two questions. Where is the middle? That is central tendency: the mean mu for `Num`, the mode (the commonest symbol) for `Sym`. How far do values stray from that middle? That is dispersion: the standard deviation for `Num`, the entropy for `Sym`. The next two sections build those four answers, one value at a time.

Two more words before we start. A **pdf** (probability density function) scores how likely each value is; its familiar picture is the bell curve. A **cdf** (cumulative distribution function) reports what fraction of the data sits at or below v. Whatever the units, a cdf runs 0..1. Remember that; it is about to earn its keep.

Center and spread, one value at a time

For numbers we track the mean and the standard deviation. The textbook formula for variance wants two passes and a stored list. Welford's update (Welford 1962) wants neither. Keep three slots (n, mu, m2). On each new value v, let d = v - mu. Then mu moves by d/n, and m2 grows by d times the *new* gap (v - mu). The standard deviation is (m2 / (n - 1)) raised to 0.5, on demand.

```
def welford(num, v): # one-pass update of mu and m2
  num.n += 1; d = v - num.mu; num.mu += d / num.n
  num.m2 += d * (v - num.mu)
```

One pass. No stored raws. Constant memory. Symbols are easier. Count them:

```
def count(sym, v): # update symbol counts
  sym.n += 1; sym.has[v] = 1 + sym.has.get(v, 0)
```

The public face of both updaters is one function, `add`, which reads a summary's `it` tag, skips the "?" that marks a missing value, and hands the value to the right updater. Its code waits for Chapter 5, where it also learns to fold rows into tables. All the cleverness lives in the parts shown above.

Entropy is just counting

For a bag of symbols with counts k out of n, the entropy is the sum over symbols of -(k/n) log2 (k/n). It measures surprise: how many yes-or-no questions, on average, to name the next symbol. Take the bag "aaaabbc", so n = 7 and the counts are 4, 2, 1. The three terms are -(4/7) log2 (4/7) = 0.461, -(2/7) log2 (2/7) = 0.516, and -(1/7) log2 (1/7) = 0.401. Their sum is 0.461 + 0.516 + 0.401 = 1.379 bits.[3] In code, the two middles share one roof, and so do the two diversities. (We say "diversity", div, not "variance", since variance names sd squared and we report sd.)

[3] Purists will note we never round away the working. House rule.

```
def entropy(sym): # diversity of symbolic distribution
  f = lambda p: p*log(p,2)
  return -sum(f(n/sym.n) for n in sym.has.values() if n >
   ↪ 0)

def mid(col): # center: mean (Num) or mode (Sym)
 return col.mu if col.it is Num else
  ↪ max(col.has,key=col.has.get)
```

```
def div(col): # diversity: sd (Num) or entropy (Sym)
  return entropy(col) if col.it is Sym else (
         0 if col.n < 2 else sqrt(max(col.m2, 0) / (col.n -
          ↪ 1)))
```

The demo checks the entropy arithmetic above, then checks Welford against 10,000 samples from a unit gaussian:

```
$ python3 src/lib_eg.py cols
sym mid a ent 1.379
num mu 0.007 sd 1.002
```

Nothing is comparable until it is 0..1

In auto93, weight runs in the thousands of pounds while acceleration runs in the tens of seconds. Any distance computed on raw values would be a weight measure wearing a distance costume. Hence, before comparing, we normalize using the normal: map each number to its column's cdf, the fraction of the bell curve at or below v. We compute that with the logistic curve 1 / (1 + e^(-1.702 z)), where z = (v - mu) / sd. Why 1.702? That constant pulls the logistic onto the gaussian cdf, to within 0.01 everywhere (Camilli 1994). And unlike the error function, e^x exists in every language this code will ever be ported to.[4] Whatever the units, the result runs 0..1.

[4] An older habit rescales by the seen range, (v - lo) / (hi - lo). That wants two more slots and its own epsilon, and one wild outlier squashes everyone else into a corner. The cdf runs on what Welford already keeps.

```
def norm(col, v): # v's cdf, via logistic; 0..1 (Nums only)
  if v == "?" or col.it is Sym: return v
  z = (v - col.mu) / (div(col) + TINY)
  return 1 / (1 + exp(-1.702 * max(-3, min(3, z))))
```

A tiny epsilon guards the degenerate column whose sd is zero. Numerical hygiene of that kind (a TINY here, a max(m2, 0) there) is not fussiness. It is where AI code goes to die, quietly.

One distance, three classics

With every gap scaled to 0..1, we aggregate gaps by the Minkowski formula: the p-th root of the mean of the p-th powers (Menzies and Srinivasan 2026). One knob, three famous distances: p = 1 is city-block, p = 2 is Euclidean, and large p approaches Chebyshev (the max gap wins). For two gaps of 0.3 and 0.4 at p = 2, that is ((0.09 + 0.16) / 2) ^ 0.5 = (0.125) ^ 0.5 = 0.354. You will meet the code in Chapter 5, written out longhand inside the two distance functions. Distance sits in every inner loop this book runs, so those functions stay flat: one call to norm per gap, and nothing else.

Random streams are lab equipment

Every stochastic demo in this book starts by seeding the random stream, so every transcript reproduces. Treat the generator as lab equipment: calibrated, logged, reset between experiments. When we later port code across languages, we will go further and use Lehmer's portable generator with multiplier 16807, which is 7 * 7 * 7 * 7 * 7 = 16807, i.e. 7^5 (Park and Miller 1988). Same seed, same stream, in every language. Five printed draws then certify a port before any learner runs.

The trust test

Later chapters keep saying "X beats Y". Such talk is cheap, so we tax it. Two lists of results are called the same unless three tests all agree they differ, and one sort powers all three: differ() sorts both lists once, then Cohen's rule reads three indexes per list (middles at least 0.35 spreads apart, where the spread is the 10th- to-90th percentile stretch over 2.56, since that stretch is 2.56 sds on a gaussian) (Cohen 1988), Cliff's delta binary-searches (rank shift beyond the 0.197 threshold) (Cliff 1993; Hess and Kromrey 2004), and Kolmogorov-Smirnov merges (cdf gap over the 5% critical value 1.36 sqrt((n + m) / nm)) (Massey 1951). The last two are the standard prescription for empirical work (one effect-size test, one significance test); the first is a cheap tie-breaker that keeps variance-only wobbles from passing as victories. Watch the conjunction work on a gaussian nudged by ever-larger shifts. Shifts of 0.1, 0.3, even 0.5 standard deviations pass as same (at 0.5, Cohen and Cliff see a difference but Kolmogorov-Smirnov holds out). Only from a full standard deviation do all three agree, and only then is the pair called different:

```
$ python3 src/lib_eg.py stats
shift  differ cliffs cohen
 +0.0  False  0.00   False
 +0.1  False  0.11   False
 +0.3  False  0.21   False
 +0.5  False  0.34   True
 +1.0  True   0.56   True
 +2.0  True   0.82   True
```

Note the humility this buys. Any single test can be gamed, and p-values alone say nothing about effect size. Demanding agreement from an effect-size test and a distribution test before crying "difference" makes our later victory claims conservative. When this book says "beats", it has cleared all three bars.

Lessons sighted

NOIR and the two surviving scales; Welford's one-pass update; entropy as counted surprise; cdf normalization and epsilon hygiene; the Minkowski family; seeds as lab equipment (with **PRNG** portability via 7^5); and the conjunctive trust test. That last one is this book's referee. It sits in the substrate, before any learner, on purpose.

The Substrate

This chapter assembles the pieces: a settings file, two column types, a table, and two distances. Everything in Parts II to IV is a short function over what this chapter builds. Total cost so far, including the test rig: under 250 lines.

about.py, in full

```
#!/usr/bin/env python3 -B
"""
about.py: every knob, one place. Change a value here, or
override it on the command line (e.g. --p 1), and all
downstream code obeys. No other file defines a setting.
"""

class o:
  "Dot-access struct. Its repr prints the public slots."
  def __init__(i, **d): i.__dict__.update(**d)
  def __repr__(i):
    return "{" + " ".join(":%s %s" % (k, v)
      for k, v in sorted(i.__dict__.items())
      if k[0] != "_") + "}"

the = o(
  seed = 1234567891,        # every random stream starts
 ↪  here
  p    = 2,                 # minkowski coefficient
  few  = 128,               # sample size for cheap guesses
  stop = 32,                # min rows before a split halts
  file = "data/auto93.csv") # default table (via MOOT)
```

That is the entire configuration system. To say that another way: every experiment in this book is fully described by (a) this file, (b) any –key=val overrides, and (c) one seed. When Chapter 16 hands this repository to an AI colleague, that property will matter a great deal.

Columns know their jobs

The first csv row names the columns, and the names carry the schema. An uppercase first letter means numeric. A trailing "+" or "-" marks a goal to maximize or minimize. A trailing "X" means ignore. Everything else is an observable x column. This is **CoC** (convention over configuration): the data describes itself, and no schema file exists to drift out of date. It is also schema *on read*: types come from data, not declarations.

`Tbl` reads the name row, builds one summary per column, splits the summaries out by role (all, x, y), and carries a rows list plus a `mid` slot (the table's center row, cached; adding a row clears it) that later chapters fill:

```
def Tbl(src): # first row names columns; rest is data
  src   = iter(src)
  names = next(src)
  all   = [Col(s, at) for at, s in enumerate(names)]
  cols  = o(names=names, all=all,
            x=[c for c in all if c.name[-1] not in "X+-"],
            y=[c for c in all if c.name[-1] in "+-"])
  return adds(src, o(it=Tbl, rows=[], cols=cols, mid=None))
```

Chapter 4 promised `add`'s code. Here it is, now with its full reach: a value into a column, a row into every column, a row into a table (which also clears the cached `mid`):

```
def add(i, v): # fold value into col, row into tbl
  if i.it is Tbl:
    i.rows += [v]; i.mid = None
    for col in i.cols.all: add(col, v[col.at])
  elif v != "?":
    (count if i.it is Sym else welford)(i, v)
  return v
```

Two details deserve a look. Firstly, `Num`, `Sym`, and `Tbl` are plain functions returning `o` structs tagged with an `it` slot, and one `add` serves all three (`mid` and `div` read the two column kinds). That is duck typing doing the work inheritance is usually hired for: three types, one shared verb, no class hierarchy. Secondly, `add` on a table folds the row into every column summary incrementally. The table never recomputes. It only ever updates.

```
$ python3 src/lib_eg.py tbl
rows 398 |x| 4 |y| 3
Mpg+ mu 23.84 sd 8.34
```

Read that transcript against the axiom of Chapter 1. The table has 398 rows, and 4 of its 8 columns are cheap x values while 3 are dear y goals. Everything downstream respects that boundary: x is free to read, y is metered.

`clone` deserves its one line of fame. Given any table and any subset of rows, it re-learns a fresh summary over just those rows, using the same header. Clusters, tree leaves, and sliding windows are all, underneath, clones.

```
def clone(tbl, rows=[]): # same header, fresh summaries
  return Tbl([tbl.cols.names] + rows)
```

Distance to heaven

Now the two workhorses. `distx` measures how *different* two rows are, reading only x columns, normalizing each gap to 0..1 and handling "?" by assuming the worst case (an unknown value is scored as far away as it could be, which keeps missingness from faking similarity). `disty` measures how *good* one row is: the distance from its goal values to heaven, where heaven is 0 for a minimize column and 1 for a maximize column. Zero is best.

```
def disty(tbl, row): # goal gap to heaven; 0=best
  d,n = 0,TINY
  for col in tbl.cols.y:
    if (v := row[col.at]) != "?":
      d, n = d + abs(norm(col, v) - col.heaven)**the.p, n+1
  return (d / n) ** (1 / the.p)
```

This paragraph is the definition of **distance to heaven**: collapse many objectives to one scalar by measuring each goal's normalized gap to its ideal, then aggregating with Minkowski. No weights to tune. All later chapters cite this paragraph rather than redefine it. Note also what `disty` reads: y columns only. Hence counting calls to `disty` counts the labels we bought, and every budget claim in this book is auditable from that one chokepoint.

Sort all 398 cars by `disty` and print the five best, then the five worst:

```
$ python3 src/lib_eg.py dist
Clndrs  Volume  HpX  Model  origin  Lbs-  Acc+  Mpg+  disty
     4      90   48     78       2  1985  21.5    40  0.074
     4      90   48     80       2  2085  21.7    40  0.087
     4      85   65     81       3  1975  19.4    40  0.087
```

```
4      97   52     82       2  2130  24.6    40  0.092
4      79   58     77       2  1825  18.6    40  0.095

8     440  215     73       1  4735    11    10  0.953
8     455  225     70       1  4425    10    10  0.954
8     440  215     70       1  4312   8.5    10  0.956
8     454  220     70       1  4354     9    10  0.956
8     455  225     73       1  4951    11    10  0.957
```

Notice the shape. Light, late, high-mpg cars float to the top. Big old guzzlers sink. No learner has run yet. This is just geometry over one table, and already it ranks a car lot. The whole of Part III is a set of strategies for reaching those top rows while paying for as few `disty` calls as possible.

What the substrate buys

Here ends Part I. We close with the claim the next eleven chapters must now earn. Clustering will be grouping by distx. Nearest-neighbor prediction will be distx with no fit step. Bayes will be grouping with labels. Trees will be grouping, recursively. Anomalies will be rows far from their group. Each arrives in roughly twenty lines, because the hard parts (summaries, normalization, missing values, distance, statistics) already live here, written once. The good news is that you have now read the hard parts. The bad news is two-fold: the substrate is dull, and we made you read it anyway. It was that or repeat it eleven times.

Lessons sighted

SSOT again (about.py, in full, as the experiment record); **CoC** and schema-on-read via header suffixes; duck typing, now defined; incremental summaries and clone; worst-case handling of missing values; **distance to heaven**, defined above; labels metered at one chokepoint. Onward to Part II, where the Fortune Teller takes the first appointment.

References

Brown, Amy, and Michael DiBernardo, eds. 2016. *500 Lines or Less: Experienced Programmers Solve Interesting Problems*. Lulu.

Buse, Raymond P. L., and Thomas Zimmermann. 2012. "Information Needs for Software Development Analytics." *Proc. ICSE*, 987–96.

Camilli, Gregory. 1994. "Teacher's Corner: Origin of the Scaling Constant d = 1.7 in Item Response Theory." *Journal of Educational and Behavioral Statistics* 19 (3): 293–95.

Clancey, William J. 1985. "Heuristic Classification." *Artificial Intelligence* 27 (3): 289–350.

Cliff, Norman. 1993. "Dominance Statistics: Ordinal Analyses to Answer Ordinal Questions." *Psychological Bulletin* 114 (3): 494–509.

Cohen, Jacob. 1988. *Statistical Power Analysis for the Behavioral Sciences*. 2nd ed. Lawrence Erlbaum.

Gabriel, Richard P. 1996. *Patterns of Software: Tales from the Software Community*. Oxford University Press.

Glass, Robert L. 2002. *Facts and Fallacies of Software Engineering*. Addison-Wesley.

Hess, Melinda R., and Jeffrey D. Kromrey. 2004. "Robust Confidence Intervals for Effect Sizes." *Annual Meeting of the American Educational Research Association*.

Hunt, Andrew, and David Thomas. 1999. *The Pragmatic Programmer*. Addison-Wesley.

Knuth, Donald E. 1984. "Literate Programming." *The Computer Jour-*

nal 27 (2): 97–111.

Lions, John. 1996. *Lions' Commentary on UNIX 6th Edition, with Source Code*. Peer-to-Peer Communications.

Massey, Frank J. 1951. "The Kolmogorov-Smirnov Test for Goodness of Fit." *Journal of the American Statistical Association* 46 (253): 68–78.

Menzies, Tim, and Srinath Srinivasan. 2026. "Can AI Be Easy? Lessons Learned from the EZR.py Toolkit." *Software: Practice and Experience*.

Norvig, Peter. 1992. *Paradigms of Artificial Intelligence Programming: Case Studies in Common Lisp*. Morgan Kaufmann.

Ousterhout, John. 2018. *A Philosophy of Software Design*. Yaknyam Press.

Park, Stephen K., and Keith W. Miller. 1988. "Random Number Generators: Good Ones Are Hard to Find." *Communications of the ACM* 31 (10): 1192–201.

Pearl, Judea, and Dana Mackenzie. 2018. *The Book of Why: The New Science of Cause and Effect*. Basic Books.

Schreiber, Guus, Hans Akkermans, Anjo Anjewierden, et al. 2000. *Knowledge Engineering and Management: The CommonKADS Methodology*. MIT Press.

Seibel, Peter. 2005. *Practical Common Lisp*. Apress.

Spinellis, Diomidis. 2003. *Code Reading: The Open Source Perspective*. Addison-Wesley.

Stevens, S. S. 1946. "On the Theory of Scales of Measurement." *Science* 103 (2684): 677–80.

Welford, B. P. 1962. "Note on a Method for Calculating Corrected Sums of Squares and Products." *Technometrics* 4 (3): 419–20.

Wilson, Greg. 2024. *Software Design by Example: A Tool-Based Introduction with Python*. CRC Press.